

AI-Driven Search over PhishLLM Configurations for Reference-Based Phishing Detection

Nazish

CS7602 - AI-Driven Research for Security

MBZUAI

nazish.baliyan@mbzuai.ac.ae

Abstract—PhishLLM is a recent phishing detector that uses reference-based reasoning rather than a fixed list of known brands. For this course project, I do not attempt a full reproduction of the paper’s large-scale deployment. Instead, I cast the midterm as a tighter search problem: keep the PhishLLM-style pipeline fixed, and use AI to search over inference-time choices such as prompt templates, validation rules, thresholds, interaction depth, and final fusion logic. A candidate is a fully specified JSON configuration; the evaluator runs it on a frozen 400-site validation split and returns precision, recall, F1, false-positive/false-negative rates, median runtime, estimated cost per 1K pages, and failure buckets. The hard contract is explicit: precision ≥ 0.95 , median runtime ≤ 6.0 sec/page, estimated cost $\leq \$12$ per 1K pages, and a deterministic evaluation environment with cached OCR/caption outputs and fixed prompts. The design is intentionally rubric-aligned: it defines a concrete candidate space with full examples, a repeatable `evaluate(candidate, split)` contract, an iterative AI search loop with a real proposer prompt, explicit scope limits, and visible prototype evidence.

I. PAPER CONTEXT AND THREAT MODEL

A. Paper Context

Phishing detectors try to determine whether a webpage impersonates a legitimate brand in order to steal credentials. The central claim of *PhishLLM* is that phishing detection should not depend on a pre-defined reference list of brands. Instead, the system combines visual evidence, textual evidence, and interaction flow to infer the likely target brand and decide whether the page is malicious. The paper’s pipeline has four stages: (1) infer the target brand from logo OCR and logo captioning, (2) detect whether the current page is credential-requiring (CRP), (3) if not, navigate to likely login UI elements using a transition model, and (4) declare phishing using brand-domain inconsistency, hosting-domain logic, and popularity validation. On the benchmark discussed in the paper, PhishLLM reports precision 1.0000, recall 0.7501, and median runtime 3.2 seconds per site, while also reporting substantially more verified phishing sites than prior baselines in the field study [1].

B. Threat Model

The attacker is a credential-stealing phisher who can fully control webpage text, images, HTML, layout, and hidden instructions embedded in the page. The attacker may also adapt to known detector behavior, for example by hiding login widgets, manipulating brand hints, or inserting prompt-injection text. However, the attacker cannot alter the deployed detector

itself. PhishLLM also assumes many phishing sites are hosted on attacker-controlled or hosting-service domains rather than compromised high-reputation domains. This matters because popularity validation is useful for filtering many false alarms, but it can also miss phishing pages hosted on already-indexed legitimate domains [1].

C. My Focus Slice

My project focuses on one well-scoped extension point: **AI-driven search over PhishLLM inference-time configurations**. I will not redesign the full architecture, retrain the CRP transition model, or reproduce the paper’s full field deployment. Instead, I ask: *Which prompts, thresholds, validation rules, and fusion policies give the best recall/F1 under a strict precision floor and bounded runtime/cost?* This is the right slice for CS7602 because it is faithful to the paper, security-relevant, automatically evaluable, and feasible on a midterm timeline.

II. PROJECT PROBLEM

A. Precise Objective

Given a fixed labeled set of webpage screenshots, URLs, and HTML pages, search over PhishLLM prompt/policy configurations to maximize phishing-detection recall and F1 subject to:

- precision ≥ 0.95 ,
- median runtime ≤ 6.0 sec/page, and
- estimated cost $\leq \$12$ per 1K pages.

Formally, for candidate c , the project seeks high-value points on the validation frontier

$$\max_c (\text{Recall}(c), \text{F1}(c)) \quad \text{s.t.} \quad \text{Precision}(c) \geq 0.95.$$

The frozen midterm protocol is also numeric rather than approximate: validation split `val_midterm_v1` contains **400 sites total**, stratified once as **200 phishing + 200 benign**, sampled with seed 7602, saved as `splits/val_midterm.csv`, and reused for every candidate. The midterm search budget is **10–12 evaluated candidates**; the final-project target is **20–40 candidates** across 4–5 rounds.

TABLE I
CANDIDATE SCHEMA FOR PHISHLLM CONFIGURATION SEARCH.

Field	Allowed values / default meaning
name	unique identifier for one configuration
backend	openai-gpt35, cached-replay, or local backend
brand_prompt	target-brand prompt template ID
crp_prompt	credential-page prompt template ID
use_logo_ocr / use_logo_caption	modality toggles for brand inference
prompt_defense	XML-style instruction/data separation on or off
popularity_validation	google-indexed, cached, or disabled
hosting_rule	strict or relaxed handling of hosting-service domains
max_interactions	CRP transition hops; {0,1} for the midterm
brand_confidence_min	confidence floor for reporting a brand
report_policy	final fusion logic such as mismatch_and_crp
temperature	fixed at 0.0 for repeatability
runtime_budget_sec	per-page runtime cap; default 6.0

B. Success Criteria

A successful midterm deliverable must satisfy four concrete checks:

- 1) run a deterministic `evaluate(candidate, split)` end to end on the frozen split;
- 2) compare at least three concrete candidates, including the official-style baseline;
- 3) emit structured outputs (`metrics.json`, per-sample CSV/logs, and search history) that can drive later AI search rounds; and
- 4) preserve a reproducible experiment contract: fixed split, fixed prompts, fixed backend label, and cached OCR/caption outputs.

C. Why the Problem Is Non-Trivial

This search problem is difficult for four reasons. **Asymmetric error cost:** false negatives miss attacks, but false positives waste analyst time and reduce trust. **Pipeline coupling:** a permissive brand prompt or relaxed hosting rule may improve recall but create downstream false positives. **Adversarial fragility:** removing prompt-defense structure or popularity checks may help on some pages while making the detector more vulnerable to prompt injection or misleading text. **Resource trade-offs:** PhishLLM depends on LLM reasoning and external validation, so stronger detection can increase latency and cost rather than simply improving quality [1], [2].

III. CANDIDATE REPRESENTATION, EVALUATOR, AND METRICS

A. Candidate Representation

A candidate is a structured JSON configuration that changes only *inference-time* behavior. Table I defines the core schema.

Figure 1 makes the candidate space explicit with three fully specified seed JSONs used in Round 0.

The fourth seed is **C3 low-cost:** backend cached-replay, use_logo_caption=false, popularity_validation=cached, max_interactions=0, and runtime_budget_sec=4.5. Its role is to anchor

Algorithm 1 Evaluator contract (simplified)

Require: candidate c , frozen split D
1: initialize tp, fp, tn, fn , runtime/cost accumulators, failure buckets
2: **for all** samples $s \in D$ **do**
3: read $s.url, s.html, s.screenshot$
4: $pred \leftarrow \text{BACKENDPREDICT}(s, c)$
5: update confusion matrix using true label vs. predicted label
6: update runtime, estimated cost, and per-sample logs
7: bucket errors such as CRP miss, alias false positive, or API failure
8: **end for**
9: compute precision, recall, F1, FPR/FNR, median runtime, cost per 1K
10: write `metrics.json`, `predictions.csv`, `candidate.json`, and `run_manifest.json`
11: **return** metrics dictionary and per-sample rows

TABLE II
FROZEN EVALUATOR PROTOCOL AND REPEATABILITY CONTROLS.

Control	Fixed value for the midterm contract
Split ID	val_midterm_v1: 400 sites = 200 phishing + 200 benign; sampled once with seed 7602; reused for all candidates
Backend pin	one repo commit and one backend label for the full run family; exact API model string captured at run start and frozen afterward
Determinism	temperature=0.0; cached OCR/caption outputs keyed by site ID; fixed prompt template files
Timeout / retry	8 sec/page hard timeout, 1 retry on transient API failure, then bucket as <code>api_or_parser_failure</code>
Stored artifacts	<code>metrics.json</code> , <code>predictions.csv</code> , <code>candidate.json</code> , and <code>run_manifest.json</code>

the low-cost end of the frontier. Importantly, the midterm search space now has no interaction-count ambiguity: `max_interactions` is always either 0 or 1; there is *no* “extra” second interaction in the midterm setting.

B. Evaluator Design

The evaluator is the fixed contract of the project:

```
evaluate(candidate, split) -> metrics_dict
```

Inputs are: (i) the frozen split, (ii) one candidate JSON, (iii) fixed environment settings, and (iv) cached OCR/caption outputs whenever possible. For each sample, the evaluator reads URL, HTML, and screenshot metadata, runs the selected PhishLLM-style backend, records the predicted label/brand, runtime, and estimated cost, then compares them to ground truth. The current prototype already writes three core files per run: `metrics.json`, `predictions.csv`, and a copy of the evaluated candidate configuration. For the graded evaluator, I additionally require a small `run_manifest.json` that records split ID, backend label, exact model/API identifier, cache mode, and prompt versions used in the run.

The split uses the same folder-per-site layout as the official artifact, plus a `labels.csv` file containing `site_id`, `label`, `target_brand`, `is_crp`, and optional notes. Because the split is frozen once and reused exactly, `evaluate(candidate, split)` remains a stable contract instead of a moving target.

C. Metrics and Reporting

The evaluator reports metrics that directly match the phishing-detection threat model:

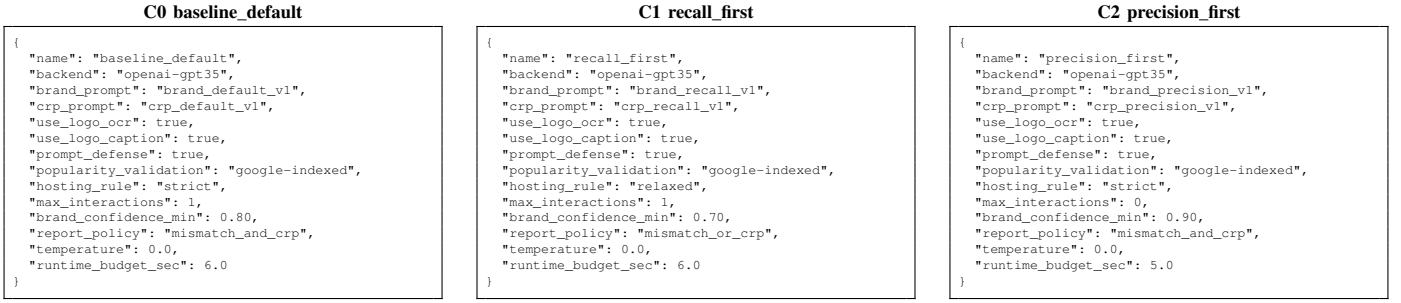


Fig. 1. Three fully specified seed candidates. Each example instantiates every field in Table I, which removes ambiguity about defaults and search-space limits.

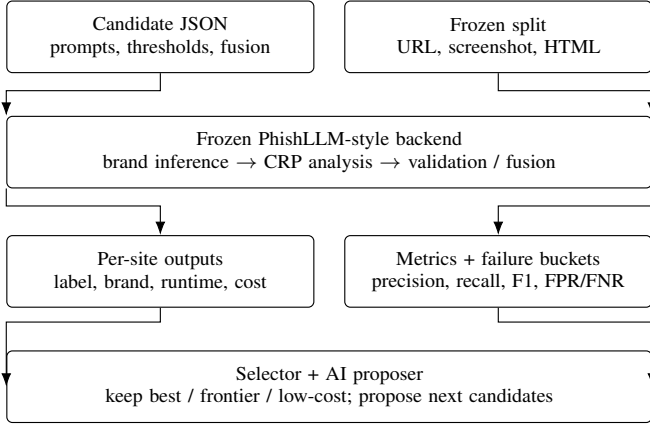


Fig. 2. Search-over-configuration architecture. The evaluator keeps the backend fixed and varies only prompts, validation, thresholds, and fusion policies.

- **Precision, recall, F1:** primary quality metrics; recall is central because the paper’s main claim is improved phishing coverage [1].
- **False-positive rate and false-negative rate:** operationally important for analyst trust and missed attacks.
- **Median runtime per page:** required because practical detector latency matters and the paper reports runtime explicitly [1].
- **Estimated cost per 1K pages or equivalent API usage:** tracks practical deployment cost [2].
- **Failure buckets:** brand hallucination, CRP miss, alias false positive, prompt-injection failure, and API/parser failure. Full official integration can add hidden-login-page miss when the transition model is exercised.

The main selection rule is strict: discard any candidate with precision below 0.95, median runtime above 6.0 sec/page, or estimated cost above \$12 per 1K pages; then rank the remaining candidates by recall, break ties by F1, and prefer lower runtime/cost. I also preserve a small Pareto frontier over {precision, recall, runtime} to avoid collapsing too early to one point.

IV. AI-DRIVEN SEARCH STRATEGY

A. Round Structure and Feedback

This project uses AI as a true iterative search process, not a one-time assistant. Each round has five stages: candidate storage, evaluator execution, feedback summarization, AI

proposal, and pruning/selection. After each round, the system produces a compact feedback object containing aggregate metrics, confusion matrix, top false-positive and false-negative examples, average runtime/cost, and failure-bucket counts.

B. Actual Proposer Prompt and Diversity Rule

The proposer receives the objective, hard constraints, candidate schema, top candidates so far, one diverse unsuccessful candidate, and the latest error summary. A concrete meta-prompt template is shown in Listing 1.

Listing 1. Concrete proposer prompt template used after each evaluation round.

```

Task: Propose 4-6 schema-valid PhishLLM candidate JSONs.
Objective: maximize recall and F1 on val_midterm_v1.
Hard constraints:
- precision >= 0.95
- median_runtime_sec <= 6.0
- estimated_cost_per_1k <= 12.0
- max_interactions in {0,1}
- temperature = 0.0
Inputs provided to you:
- candidate schema
- top 2 current candidates
- one diverse failed candidate
- failure-bucket summary and confusion matrix
Output requirements:
- JSON array only
- one-sentence hypothesis per candidate
- no normalized-hash duplicates
- prefer edits to prompts, validation, threshold, and fusion only

```

A candidate counts as *diverse* only if it differs from the current best on at least **two** of the following fields: brand_prompt, crp_prompt, prompt_defense, popularity_validation, hosting_rule, max_interactions, threshold bucket, or report_policy. This makes the exploration rule concrete instead of subjective.

C. Validation, Selection, and Stopping

Round 0 starts from the four seed candidates C0–C3. Every later proposal passes three filters before selection: (i) schema validity, (ii) duplicate removal via normalized config hash, and (iii) hard-limit checks after evaluation. Among valid candidates, I keep the top two by the main ranking rule, up to two diverse frontier candidates, and one low-cost candidate. The search stops when any of the following happens: (1) five rounds have completed, (2) the Pareto frontier stops improving for two consecutive rounds, or (3) the best recall under the precision floor improves by less than 0.01 for two rounds. If the full AI proposer slips, the fallback still preserves the project contract: manually define 6–8 strong candidates, run the same fixed evaluator, and log the intended next-round AI prompt for completeness.

V. SCOPE, ASSUMPTIONS, AND PROTOTYPE EVIDENCE

A. Scope Boundaries

Included scope: inference-time search over prompts, thresholds, validation, and fusion policies on a reduced benchmark. Excluded scope: retraining the CRP transition model, reproducing the full field deployment, building a new phishing detector from scratch, or collecting a new 90K-site crawl. These exclusions are reasonable because the rubric rewards a tight, implementable project contract rather than an over-scoped reimplementaion.

B. Assumptions and Limitations

The evaluator assumes static ground-truth labels and a frozen dataset during validation. It also assumes the operator wants high-precision analyst-facing reporting, so precision is treated as a constraint rather than a secondary metric. The official reproduction path assumes API access for OpenAI and Google-backed validation; if that is unavailable, cached replay or a local backend becomes the fallback. Results will be limited to the chosen split and the fixed deployment settings used during evaluation.

C. Visible Prototype Evidence

The submission materials already move beyond a purely speculative design. The uploaded evaluator script computes confusion-matrix metrics, runtime, cost-per-1K estimates, and failure buckets; the companion search-loop script stores round-wise histories, meta-prompts, and ranked top candidates. To make this visible inside the PDF, Listing 2 shows a *toy smoke-test artifact* produced from the current harness style on a 4-site mock dataset. It demonstrates that the evaluator contract runs end to end, but it is **not** a benchmark result.

Listing 2. Toy smoke-test artifact from the prototype harness (evidence only, **not a benchmark result**).

```
metrics.json excerpt
{
  "candidate_name": "baseline_default",
  "num_samples": 4,
  "precision": 1.0,
  "recall": 1.0,
  "f1": 1.0,
  "median_runtime_sec": 3.5,
  "estimated_cost_per_1k": 3.5,
  "tp": 2, "fp": 0, "tn": 2, "fn": 0
}

predictions.csv header
site_id,true_label,pred_label,target_brand,pred_brand,
runtime_sec,estimated_cost,reasons,crp_reason
```

VI. FEASIBILITY, RISKS, AND WORK PLAN

A. Concrete Budget and Deliverables

The midterm path is intentionally modest: use the official repository as the baseline system, wrap it with a thin evaluator, and search over JSON-level configuration knobs. Using the paper’s reported median runtime of 3.2 sec/site as a rough baseline, one 400-site candidate evaluation is about 21 minutes before API variability; four seed candidates are therefore around 1.4 hours, and a 10–12 candidate midterm search is on the order of a few hours rather than days [1].

TABLE III
FROZEN MIDTERM BUDGET AND ARTIFACTS.

Item	Fixed value / artifact
Validation split	400 sites total, reused for every candidate; 200 phishing + 200 benign
Hard caps	precision ≥ 0.95 ; median runtime ≤ 6.0 sec/page; estimated cost $\leq \$12$ per 1K pages
Midterm search scale	4 seed candidates + 6–8 additional candidates across 1–2 search rounds
Final-project target	20–40 evaluated candidates across 4–5 rounds
Saved outputs	metrics.json, predictions.csv, candidate.json, run_manifest.json, round-wise meta-prompts, and ranked search logs

B. Top Risks and Mitigations

- **API instability or quota failure.** Mitigation: cached replay mode, smaller split, or local backend fallback.
- **Official pipeline runs slowly.** Mitigation: keep the runtime cap at 6.0 sec/page, freeze the 400-site split, and restrict the midterm search to 1–2 rounds.
- **Search automation slips.** Mitigation: keep the evaluator fixed and evaluate a manual grid of 6–8 schema-valid candidates; still log the intended AI prompt and next-step policy.
- **Evaluator noise or parser brittleness.** Mitigation: temperature 0, cached OCR/caption outputs, fixed prompts, 1 retry, and stored raw logs for auditability.

C. Two-Day Midterm Plan

- 1) **Day 1 morning:** verify the official environment or cached prototype path and run one smoke test.
- 2) **Day 1 afternoon:** freeze `val_midterm_v1`, finalize `labels.csv`, and finish the evaluator wrapper.
- 3) **Day 1 evening:** run seed candidates C0–C3 and verify precision/recall/F1/runtime logging.
- 4) **Day 2 morning:** run one or two AI-search rounds (or the manual fallback grid) over 6–8 additional candidates.
- 5) **Day 2 afternoon:** export the top-candidate table, FP/FN error summary, search logs, and one artifact for the presentation.

Expected evidence: a baseline-vs-variants comparison table, one error-bucket summary, one search-history artifact, and one embedded proof that `evaluate(candidate)` ran end to end.

VII. CONCLUSION

This midterm is strongest when framed not as “build a brand-new phishing detector,” but as a disciplined search problem over *PhishLLM configurations*. The revised report now makes the candidate space fully explicit, freezes exact numeric constraints, tightens evaluator repeatability, shows a real AI proposer prompt, and embeds visible prototype evidence. That makes the document closer to a full-marks project contract: specific enough to implement, narrow enough to finish, and strong enough to extend into the final project.

TABLE IV
ROUND-0 SEED SET AND INTENDED SEARCH COVERAGE.

Seed	Main edits vs. baseline	Intended role / hypothesis
C0 baseline_default	official-style anchor	control point for comparison and evaluator sanity checking
C1 recall_first	lower threshold, relaxed hosting rule, recall-oriented prompts, <code>mismatch_or_crp</code>	catch harder CRP pages and non-obvious phishing flows at some FP risk
C2 precision_first	higher threshold, 0 hops, stricter prompts, <code>mismatch_and_crp</code>	minimize false alarms and enforce analyst-grade precision
C3 low_cost	cached replay, cached validation, no logo caption, 0 hops	probe the low-latency / low-cost edge of the search frontier

REFERENCES

- [1] R. Liu, Y. Lin, X. Teoh, G. Liu, Z. Huang, and J. S. Dong, "Less Defined Knowledge and More True Alarms: Reference-based Phishing Detection without a Pre-defined Reference List," in *33rd USENIX Security Symposium (USENIX Security 2024)*, 2024.
- [2] Duanexiao, "PhishLLM GitHub Repository," GitHub. [Online]. Available: <https://github.com/Duanexiao/PhishLLM>